

Module 4

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 4 Study Guide and Deliverables

February 6 – February 12

Theme:	Authentication and Authorization
Topics:	• Something you know, you are, and you have: password, biometrics, token. • Digital signature and Kerberos • Digital certificate and PKI • Access policy, access control matrix, access control list, capability, RBAC
Readings:	• Chapter 2.1 and Chapter 2.2
Assignments:	• Assignment 2 due Tuesday, February 13 at 6:00 AM ET • Lab 4 due Tuesday, February 13 at 6:00 AM ET
Assessments:	• Quiz 4 due Tuesday, February 13 at 6:00 AM ET
Discussions:	• Discussion 2 posts due Tuesday, February 13 at 6:00 AM ET. Any posts after the due time will not be included in the grading process.
Live Classrooms:	• Lecture: Tuesday, February 6, 7:30-9:00 PM ET • Facilitator office hour: TBD

■ Topic 1 Authentication

Learning Outcomes

By the end of this module, you will be able to:

- Describe basic concepts related to authentication and authorization such as dictionary attack, brute force attack, salted hash, multifactor authentication, Kerberos, single sign on, delegated authentication, ACL, capability list, mandatory access control, discretionary access control, reference monitor, RBAC, BLP model, Biba model, etc.
- Understand the rationale behind password policy and the tradeoff between password complexity and usability.
- Understand what the server certificate is, and how it is validated based on PKI.
- Interpret an access control matrix.
- Know how to determine if a protection system is mandatory or discretionary.
- Know how Unix mode bits work.

Module Introduction

The basis of computer security is access control, i.e., whether someone is authorized to take some action on something. We will discuss access control (authorization) later in this module. To enforce access control, we must first identify who someone is, which is authentication. Authentication mechanisms use any of the following three qualities to confirm a user's identity:

- Something you know, e.g., passwords, PIN numbers, mother's maiden name, etc.
- Something you are, e.g., fingerprint, face, voice, etc.
- Something you have, e.g., identity badges, physical keys, etc.

Something You Know

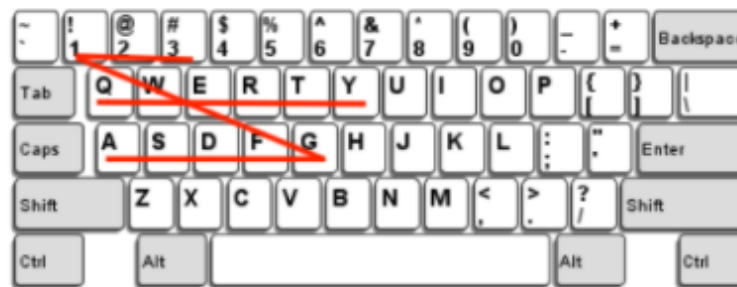
Password Use

A password is a straightforward way to authenticate users. Think about your own experience. You are often asked to enter some piece of identification, e.g., a name or an assigned user ID. Because such identification might be publicly available or easy to guess, you also need to enter a password. If your entered password matches the one on file, you are authenticated and allowed access. Otherwise, you have the chance to enter it again in case you mistyped it.

Common Password Problems

Simple passwords

Password protection sounds like a relatively secure system to authenticate users, but user practice degrades its quality. The well-known dilemma of passwords is finding the balance between security and usability. For a password to be unguessable, it should be as random as possible. However, a random password is hard for users to memorize. Usually, to remember a password and use it whenever needed, users integrate some kind of pattern into it to make it memorable, e.g., a birthday, an address, an English word, a zigzag based on keyboard layout, etc. Commonly used passwords that contain such patterns include “password,” “user,” “abc123,” “aaaaa,” “asdfg123,” “qwerty,” “123456,” etc. Such specific patterns, which decrease the uncertainty of passwords, can be leveraged by attackers to guess passwords efficiently.



Zigzag Pattern on Keyboard (qwerty and asdfg123)

Password reuse across multiple accounts

Users may have multiple accounts, each requiring a password to authenticate users. Even if a user chooses a complex (hard-to-predict) password and successfully memorizes it, it will be quite difficult for that user to have different passwords with such complexity for multiple accounts and memorize all of them. Instead, most users simply reuse the same password across different accounts, or revise a little bit of one password to obtain another. One consequence of this affects usability, i.e., later, as users have many variants of the passwords across different accounts, they cannot recall which variant is for which account. The other consequence directly impacts security. Among all the accounts that the user has, there could be banking/financial accounts, as well as online forum and blog accounts. As you can imagine, the security controls deployed for the forum/blog servers and banking/finance accounts could be significantly different due to their different investments in security. Hence, it will be relatively easier for attackers to compromise forum/blog servers and obtain users' passwords on file. From there, the attackers are able to compromise users' banking/finance accounts easily if they use the same or similar passwords across different accounts.

Use password manager

Password manager software is a recommended solution for users, since it fulfills both usability and security simultaneously, e.g., [1Password](#), [LastPass](#), [Keychain](#), [Chrome autofill](#), etc., are one recommended solution for users, since they fulfill both usability and security simultaneously. They are either browser extensions or separate applications. These software managers can create complicated passwords on behalf of users, to fulfill the password complexity requirements, thus increasing the difficulty of password guessing. Meanwhile, they store

such passwords (mostly in encrypted format) and automatically fill them in whenever needed for authentication purposes, thereby removing users' burden to memorize them. However, a recent study shows that users are reluctant to use password managers or feel uncomfortable exposing their passwords to password managers. For more details, refer to the paper "[A Usability Study and Critique of Two Password Managers](#)".

Writing passwords down

Users may consider writing their passwords down on paper, which they try to keep securely. However, this poses the risk of shoulder surfing attacks, as attackers could look over the users' shoulder while they are writing down their passwords. Afterwards, users must always keep the paper to themselves to avoid leaking the passwords incidentally. Furthermore, usability might also be a problem. For instance, locking the paper in a safe box can surely secure the paper, and therefore, the passwords, but it is not an ideal solution since users may need the paper (and passwords) even when they are away from home.

Dictionary Attack

The following are 12 password guessing techniques listed in our textbook that an attacker might try, in descending order, to hack passwords:

- There is no password.
- The password is the same as the user ID.
- The password is the same as, or is derived from, the user name.
- The password is on a common word list (e.g., password, secret, private), or is a common name or pattern (e.g., qwerty, aaaaaa).
- The password is contained in a short college English dictionary.
- The password is contained in a complete English word list.
- The password is contained in common non-English-language dictionaries.
- The password is contained in a short college English dictionary with capitalizations (PaSsWoRd) or substitutions (digit 0 for letter O, and so forth).
- The password is contained in a complete English dictionary with capitalization or substitutions
- The password is contained in common non-English dictionaries with capitalizations or substitutions.
- The password can be obtained by brute force, trying all possible combinations of alphabetic characters.
- The password can be obtained by brute force, trying all possible combinations from the full character set.

Eventually, every password can be guessed (till the last step), if time is not an issue. Hence, when we discuss password strength, we really mean how long, or how many guesses, are needed to finally hack it. Attackers have developed efficient ways to guess passwords more quickly, such as the dictionary attack. Site administrators usually use dictionaries, e.g., words, phrases, places, mythological names, science fiction character names, etc., to scan for weak passwords. However, such dictionaries can also be used by attackers to guess users' passwords. Some clever users might think up the idea of choosing a simple password (which might be in a

dictionary), but replacing some letters, e.g., digit 0 for letter O, digit 1 for letter L, digit 3 for letter E, @ for letter A, etc. Keep in mind that attackers can also think up such substitutions, and extend their dictionaries to launch attacks.

Attackers can always launch an exhaustive or brute force attack, i.e., trying all possible passwords, usually in an automated fashion. Depending on the complexity of the passwords, brute force attacks may take months, years, or even longer. But keep in mind that attackers will always start trying simple passwords first, as indicated in the above steps.

Password Storage

The first time users register their account, they choose a password associated with their account ID. The computer or the server needs to store the account ID and password locally in a table for later authentication purposes. Storing the account ID sounds fine, since such IDs are sometimes publicly known, as is the case for online forum/blog/social platforms, where they uniquely identify a person. However, this should not be the case for password storage. Once the computer/server gets compromised, accessing the table will give the attacker control over all the accounts easily. Instead, operating systems store the passwords in a concealed form, e.g., by hashing the passwords and storing the hash values, as shown in the right-hand table pictured below. In particular, when a user creates an account and chooses a password, the operating system immediately hashes the password and stores the hash value. When authenticating a user's login later, the operating system just accepts the user's input in the password field, computes its hash, and compares it with the stored hash. A match means a successful authentication. Such hash-based authentication makes sense due to the two properties of a good hash function. Due to collision resistance (two different inputs cannot have the same hash value), only the correct password will produce the same hash value as the stored one. Therefore, comparing the hash values functions equivalently as comparing passwords directly. Meanwhile, since the hash function is one-way, even if the hash values of the passwords are leaked, users' passwords are still confidential.

Plaintext		Hashed	
Identity	Password	Identity	Password
Jane	qwerty	Jane	0x471aa2d2
Pat	aaaaaa	Pat	0x13b9c32f
Phillip	oct31witch	Phillip	0x01c142be
Roz	aaaaaa	Roz	0x13b9c32f
Herman	guessme	Herman	0x5202aae2

Claire	aq3wm\$oto!4		Claire	0x488b8c27
--------	--------------	--	--------	------------

Password Storage Example (from [security 5ed, Pfleeger, et al, 2015])

Test Yourself

Suppose a system stores the account IDs and hash values of passwords for existing users. After a system compromise, the attacker obtains the table containing the above information (i.e., the right-hand table of the above figure). What will happen if the attacker enters “Jane” as the account ID, and “0x471aa2d2” as the password? (Actually, “0x” is the prefix to indicate “471aa2d2” is a hexadecimal number. Here, we just assume “0x471aa2d2” entirely represents the hash value of the password for simplicity.) Can the attacker log in successfully?

No. The attacker cannot log in. We use P to indicate the original password for Jane and $h()$ to indicate the hash function. Then, $0x471aa2d2 = h(P)$. When the system receives “0x471aa2d2” as input for password, it will compute the hash value of “0x471aa2d2,” since the system always assumes the plaintext password as input. Therefore, the system will compute $h^2(P) = h(0x471aa2d2)$, which means hash twice of the original password. Comparing $h^2(P)$ with $0x471aa2d2 = h(P)$ certainly will cause a mismatch, thus failing the authentication.

Storing the hash values of passwords is still not enough. As discussed previously, some users may choose easy-to-predict passwords. Accordingly, attackers can build a rainbow table, a list of the hash values of the commonly-used passwords. Searching for matching entries in the rainbow table, the attacker can learn that Pat’s password is “aaaaaa,” Roz’s password is “aaaaaa,” and Jane’s password is “qwerty.” Moreover, by examining the hash values of passwords, the attacker may find some identical values, e.g., those for Pat and Roz, from which the attacker can infer that both of them might have chosen a commonly used password. Hence, the attacker can start from usual passwords with high confidence.

Sample Rainbow Table	
Original Password	Encrypted Password
asdfg	0x023c94fc
p@55w0rd	0x04ff38d9
aaaaaa	0x13b9c32f
password	0x2129f30d

qwerty	0x471aa2d2
12345678	0x4f2c4dd8
aaaaaaa	0x8384a8c8
	etc.

(Pfleeger, et al., 2015)

Salted Hash		
Identity	ID+password (not stored in table)	Stored Authentication Value
Jane	Jan+qwerty	0x1d46e346
Pat	Pat+aaaaaaa	0x2d5d3e44
Phillip	Phi+oct31witch	0xc23c04d8
Roz	Roz+aaaaaaa	0xe30f4d27
Herman	Her+guessme	0x8127f48d
Claire	Cla+aq3wm\$oto!4	0x5209d942

(Pfleeger, et al., 2015)

To counter the rainbow table attack, an extra piece of data specific to each user, called salt, is combined with a plaintext password, and the system computes the hash value of the combination. For instance, the system computes the hash value of “Pat+aaaaaaa” for Pat, and “Roz+aaaaaaa” for Roz, thus storing different hash values even if they happen to choose the same password. Also, the attacker cannot build a rainbow table because the common passwords now all have a unique component too.

Password Policies

The security of passwords can be improved by enforcing password policies. For instance, the [SANS institute](https://www.sans.org/) provides guidelines for password construction and protection:

Strong passwords have the following characteristics:

- Contain at least three of the five following character classes:
 - Lower case characters
 - Upper case characters
 - Numbers
 - Punctuation
 - "Special" characters (e.g. @#\$%^&*()_+|~=-\`{}[]:"';<>/ etc.)
- Contain at least fifteen alphanumeric characters.

Weak passwords have the following characteristics:

- Contain less than fifteen characters
- Contain a word found in a dictionary (English or foreign)
- Contain a common usage word such as:
 - Names of family, pets, friends, co-workers, fantasy characters, etc.
 - Computer terms and names, commands, sites, companies, hardware, software.
 - The words "<Company Name>," "sanjose," "sanfran," or any derivation.
 - Birthdays and other personal information such as addresses and phone numbers.
 - Word or number patterns, such as aaabbb, qwerty, zyxwvuts, 123321, etc.
 - Any of the above spelled backwards.
 - Any of the above preceded or followed by a digit (e.g., secret1, 1secret)

Password Protection Standards

- Always use different passwords for <Company Name> accounts than for other non-<Company Name> access (e.g., personal ISP account, option trading, benefits, etc.).
- Always use different passwords for various <Company Name> access needs whenever possible. For example, select one password for systems that use directory services (i.e. LDAP, Active Directory, etc.) for authentication and another for locally authenticated access.
- Do not share <Company Name> passwords with anyone, including administrative assistants or secretaries. All passwords are to be treated as sensitive, confidential <Company Name> information.
- Passwords should never be written down or stored online without encryption.
- Do not reveal a password in email, chat, or other electronic communication.
- Do not speak about a password in front of others.
- Do not hint at the format of a password (e.g., "my family name").
- Do not reveal a password on questionnaires or security forms.
- If someone demands a password, direct them to the Information Security Department.
- Always decline the use of the "Remember Password" feature of applications (e.g., Eudora, Outlook, Netscape Messenger).

Password Recovery

Most systems provide a fail-safe password recovery option in case users forget their passwords. One mechanism used frequently to recover passwords is to answer a series of security questions, the answers of which were recorded when users created the account, e.g.:

- What is your mother's maiden name?
- What is the model of your first car?
- What is the name of your first pet?
- Which neighborhood did you grow up in?
- What is the name of your elementary school?
- What is the name of your first employer?
- Where was your father born?
- ...

However, the answers of the above questions could be learned or inferred using social engineering techniques, e.g., checking the victim's social media accounts, public profiles, etc. Another way to recover passwords is that the system sends a password recovery email to the user's backup email account. Such a recovery method requires users to still remember the password of the backup email account.

Something You Are

Biometrics are biological properties, based on some physical characteristic of the human body, such as one's fingerprints, voice, face, retina and iris, hand geometry (shape and size of fingers), handwriting, signature, hand motion, etc. The advantages of biometric authentication are obvious; biometrics cannot be lost, stolen or forgotten, and are always available.

Problems of biometric authentication

Although effective, biometric authentication also suffers from the problems below:

- Some users may resist biometrics, due to cultural beliefs, feelings of intrusion, etc.
- Biometric recognition devices can be costly, especially for companies with lots of employees.
- Biometric readers and comparisons can become a single point of failure. Forgetting a password is a user's fault, but failing biometric authentication is not.
- False positives (incorrectly confirming an identity) and false negatives (incorrectly denying an identity) need to be well balanced. Usually, reducing a false positive rate increases false negatives, and vice versa. We may think a system with a 0.001 percent false positive rate and a 0.1 percent false negative rate is good enough. (Actually, such a system is already hard to obtain.) However, if the population is large and the asset extremely valuable, such a small percentage of errors can still cause catastrophic results.

- Forgeries of biometrics are possible. For instance, fingerprints can be easily collected, and artificial fingerprints can be produced using cheap gelatin or silicone. When facial recognition techniques were first used to unlock smartphones, someone successfully unlocked a phone by using a picture of its owner on another phone. Nowadays, [some researchers have used 3D printed heads to beat modern face recognition](#) as well. As early as 2014, [a study demonstrated that tinted contact lenses can fool iris recognition system](#) into wrong decisions. Recently, due to the fast progression of deep learning, synthetic images/videos (e.g., [deepfakes](#)), or voices (e.g., [fakeyou](#)) can be generated automatically.

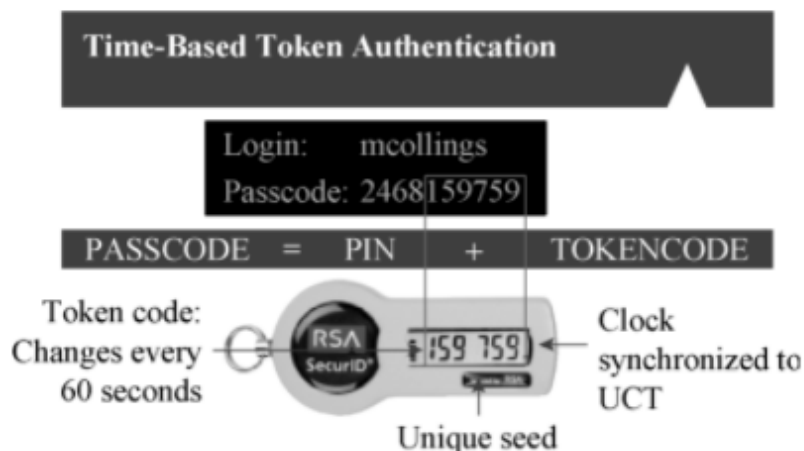




From [Fake Fingerprints](#)

Something You Have

Something you have means that you have a physical object, i.e., a token, in your possession. Examples of tokens include badge/identity cards, credit cards with a magnetic stripe, credit cards with an embedded computer chip, etc. The SecurID token from RSA lab is shown in the below figure. To use it, you enter the number currently displayed on the token together with your PIN as the passcode to pass authentication. Each token has a unique seed used to generate a series of unpredictable numbers that change every minute. Hence, each token can only be used to authenticate its assigned owner and an expired token also fails authentication. You may wonder how the computer system knows the current number displayed on the token. Does the device need to send such data to the computer via wireless connection? The trick is the unique seed. The computer system records the unique seed for each token, and performs the same computation as the token. Therefore, it can derive the same number as the token every minute to authenticate users correctly



SecurID Token from RSA Lab (from Pfleeger, et al. 2015)

Multifactor Authentication

The screenshot shows the Boston University login page. At the top left is the "BOSTON UNIVERSITY" logo. At the top right is a "Settings" button. Below the logo are three large green buttons: "Send Me a Push", "Call Me", and "Enter a Passcode". At the bottom left is a checkbox labeled "Remember me for 30 days".

Using two or more authentication methods discussed above to authenticate users is called multifactor authentication. In particular, combining two factors is called two-factor authentication. One example of two-factor authentication is the recently deployed [BU login system based on Duo security](#). First, you still need to enter your BU ID and password. Then, you are asked to further confirm your identity via text message, voice call, or a push for approval on the Duo app running on your smartphone (identified

during the initial setup). As wearable devices become more popular, smart watches can also be used to perform similar tasks as the smartphone.

Test Yourself

Multifactor authentication uses a combination of factors to authenticate users.

True

False

Test Yourself

Choose all the password policies that are good practice.

Use different passwords for different accounts

Use a combination of letters, digits, and special characters to build passwords

Do not reveal passwords in emails, blog posts, etc.

Use a shorter password, such as a simple English word, to remember it easily

Do not use zigzag patterns based on keyboard layout

Use birthdays or addresses as passwords to remember them easily

Test Yourself

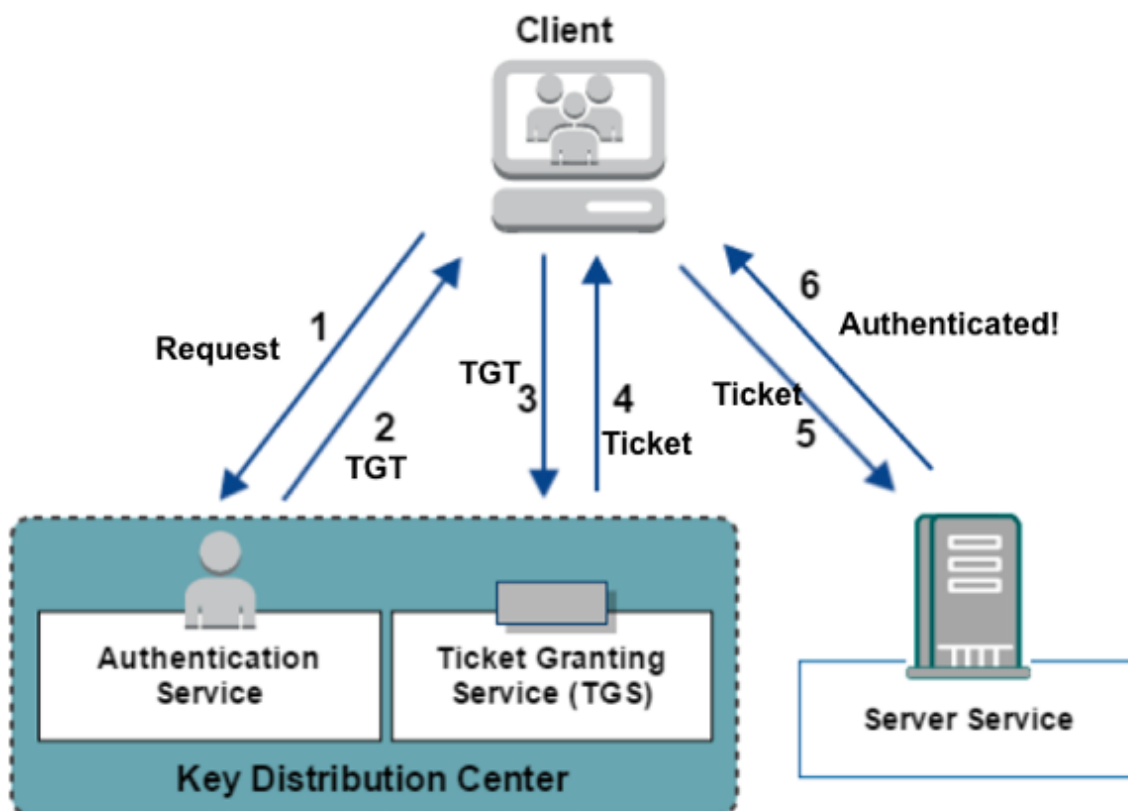
Explain what salted hash is, and what problem it can defeat.

An extra piece of data specific to each user, called salt, is combined with a plaintext password. The system then computes the hash value of the combination, and stores the salted hash. For instance, the system computes the hash value of "Pat+aaaaaa" for Pat, and "Roz+aaaaaa" for Roz, thus storing different hash values even if they happen to choose the same password. Also, an attacker cannot build a rainbow table because the common passwords now all have a unique component too.

Kerberos

[Kerberos](#) is a computer-network authentication protocol designed by MIT, mainly based on symmetric key cryptography. It is one of the most widely used (non-web) centralized password systems in existence. The BU login system is built on top of [Kerberos protocol](#). As shown in the figure below, the goal is to authenticate the client to the server service (e.g., email server, etc.). Key Distribution Center (KDC) is a trusted third party that provides two services, authentication service (AS) and ticket granting service (TGS). Each of the clients (to be authenticated) and server services (requesting client authentication) has a dedicated Kerberos password known to AS. First, the client is authenticated by AS using the password, and requests the authentication to the server service. AS sends a ticket granting ticket (TGT) back to the client, indicating the client is authenticated. After

receiving TGT, the client forwards it to TGS and requests a ticket to connect to the server service. TGS verifies the authenticity of TGT, and provides a ticket to the client for the server service. Finally, the client sends the received ticket to server service, which verifies the authenticity of the ticket, thus finishing the authentication. You may wonder what is the purpose of TGS and why doesn't AS send tickets (for the server service) directly to the client. One advantage of separating authentication and ticket granting is to facilitate single sign on (SSO). Think about the BU login system. After the first time you log in, you can access your BU email, Blackboard account, BU student link, etc. without logging in again. Note that they are different services, but you just use a single ID and password to log in to all. Also, you just log in once to access all those services, without re-entering authentication factors again for each of them. Such SSO is well-supported by Kerberos protocol. Just think about replacing the server service in the below figure with the BU email service, Blackboard service, or BU student link service. After you are authenticated by AS by entering your BU ID and password, you get a TGT from AS. Using this TGT, you can request dedicated tickets for the BU email service, Blackboard service, or BU student link service, respectively, without being authenticated by AS again.



Kerberos Protocol

Note that SSO is different from delegated authentication, which relies on another system to authenticate users. For instance, when you open the login page of Spotify, you have the options to “continue with Facebook,” “continue with Apple,” or “continue with Google” in addition to logging in directly using your Spotify account. If you choose “continue with Facebook” for example, then you will be redirected to the Facebook login page. This is a delegation example because Spotify relies on Facebook to authenticate you. Certainly, Spotify does not know your Facebook password. To be clear about the difference between SSO and delegated authentication, think about the BU login system. When deploying SSO (e.g., based on Kerberos), KDC knows your BU ID and



To continue, log in to Spotify.



CONTINUE WITH FACEBOOK



CONTINUE WITH APPLE



CONTINUE WITH GOOGLE

OR

Email address or username

Email address or username

Please enter your Spotify username or email address.

Password

Password

Please enter your password.

[Forgot your password?](#)



Remember me

LOG IN

password to authenticate you. If we use delegated authentication, e.g., relying on Google to authenticate you, you will log in using your Google account. In this scenario, Google does not know your BU ID or password. Instead, Google just authenticates you based on your Google credential.

Server Authentication

Something you know, something you are, and something you have are the mechanisms used by computer/server systems to authenticate users. In the network scenario, as end users, we also need to authenticate servers, to ensure we are really connecting to the server we desire. Think about how when you type “amazon.com” into your web browser and hit enter, you see an Amazon webpage. How can you or your computer/browser know the webpage is really rendered by Amazon’s server? Is it possible that the Amazon webpage you see actually comes

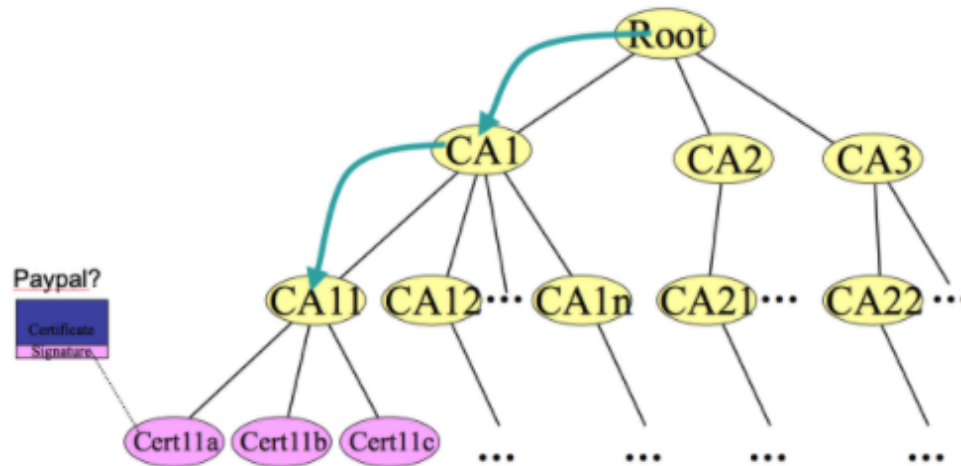
from a server controlled by an attacker? Fundamentally, how do you authenticate Amazon's server and ensure you are connecting to it? Certainly, the mechanisms used to authenticate users do not work, i.e., we cannot ask a server to register an account and maintain a password with each user. Instead, we use **certificates** to authenticate servers. (Certificates can be used to authenticate users as well.) Simply, a certificate is a binding between the server's public key and its identity, which is signed by someone called a certificate authority (CA).

Below is a segment from the certificate of [PayPal](https://www.paypal.com). Under the subject name, you can find that this certificate is issued to PayPal. The issuer is DigiCert, which is the CA. Near the bottom, you can find the public key of PayPal, and the signature from the CA. To validate this certificate, users can just use the CA's public key to decrypt the signature, thus asserting the authenticity of this certificate. If users trust the CA, then this certificate can be trusted since it is signed by the CA. Thus, users can ensure the public key on this certificate should be used by PayPal. To communicate with PayPal's servers, users use the public key on the certificate to encrypt messages, which can ensure only PayPal's servers can decrypt them since only PayPal's servers have the corresponding private key. Therefore, no matter who pretends to be a PayPal server and intercepts the encrypted messages, there is no way to decrypt the messages.

Subject Name	
Business Category	Private Organization
Inc. Country/Region	US
Inc. State/Province	Delaware
Serial Number	3014267
Country or Region	US
State/Province	California
Locality	San Jose
Organization	PayPal, Inc.
Common Name	<u>www.paypal.com</u>
Issuer Name	
Country or Region	US
Organization	DigiCert Inc
Organizational Unit	www.digicert.com
Common Name	DigiCert SHA2 Extended Validation Server CA
Serial Number	04 BD 14 DA 38 A2 BC 50 A6 C6 70 E9 A3 E1 23 6E
Version	3
Signature Algorithm	SHA-256 with RSA Encryption (1.2.840.113549.1.1.1)
Parameters	None
Not Valid Before	Thursday, April 15, 2021 at 8:00:00 PM Eastern Daylight Time
Not Valid After	Tuesday, March 15, 2022 at 7:59:59 PM Eastern Daylight Time
Public Key Info	
Algorithm	RSA Encryption (1.2.840.113549.1.1.1)
Parameters	None
Public Key	256 bytes : A1 FC 25 56 F6 A7 8E 4B 21 34 4C D6 81 05 95 7B 65 40 8C 0D 29 DD E4 20 56 18 F3 C1 97 EA DD 33 D0 73 5E 56 F7 56 15 59 75 4F 6A 1E 3F 1D 97 8C 93 B6 BA E6 DD 32 CA CC AD B4 3A A4 9C 31 DF C2 25 84 8D 93 21 29 1E F2 19 4A E5 2E 6D 73 FC 36 B6 77 2B 70 1B 9E B5 8F D5 5F B0 F2 D8 55 20 AA 3E E2 01 02 1E CC 26 E1 20 36 37 54 12 92 77 E3 84 AE 6A 93 2D 8B E8 79 C9 DF EF C2 2D C8 6A D7 A5 39 5A 1D 7A 5C A2 C8 21 98 11 28 F1 E8 54 31 B5 00 E7 CA BA F5 BE B2 84 A7 92 39 DE 4C B7 62 AD A6 7C 5E A3 A3 47 8E F7 F5 87 4F E7 25 34 66 43 A6 95 88 1C EA E5 76 DC 5D 57 3C 0D 2D D3 F9 C6 26 37 46 E1 47 78 35 CF 39 08 72 B3 76 49 AA FC 95 5B D0 B9 A4 A1 72 90 AD F8 FE A6 62 02 92 72 B6 E5 5E 82 51 50 C8 3D 9D 16 99 FC 09 54 50 B1 1B 28 1B A6 8F D5 6D 23 BA C3 63 A4 0C C1 97
Exponent	65537
Key Size	2,048 bits
Key Usage	Encrypt, Verify, Wrap, Derive
Signature	256 bytes : B4 13 C0 B1 5D 0B 40 6A 6E 84 51 BB C9 17 E2 4A 30 AC 54 DC 0F 43 35 E2 CB 25 25 EC 35 95 49 69 E3 1F 62 86 21 DD AB 54 0B 5F 3A D8 60 4D 9B 5A 91 12 C8 FF 01 8E 2A 57 CE 51 70 9C FE 7C 1E DB E1 3D 32 E3 65 9C 52 00 10 D4 A1 35 93 BE D0 E3 4E 5D D6 4F 23 81 86 F2 2C EE A8 1D 6E 56 01 48 8B F3 1B 75 58 E0 7B 8E 75 8B 24 AC CC 41 D9 1B EC 82 A1 0A 47 62 7A 0E A4 35 5E 89 DE 06 0A 90 28 54 45 00 D0 E3 0C 69 B4 D5 E9 2A BB F4 D5 40 2C 7E FD C5 0D A2 F2 5D 90 FE 0B 76 CA 50 3E 25 7F 68 74 0E 4A 1F A4 A4 0C ED F3 BA A6 FF 30 A0 6A 7D 84 FA 12 FE CB 77 09 D5 96 EB ED AE CD E8 6F 08 15 BD 8E 68 EF 0E 56 AC 88 04 43 25 93 C0 34 BE 8A 41 FF A5 EC 5C 33 94 B2 4E AF AC 81 D6 93 A2 23 59 BD 82 01 BA 6A 86 B6 81 5F 75 BF 4C BF 3A 68 CB 42 55 8D C7 E2 D2 52 50 C2 16 FA A2

Certificate of PayPal.com

Probably, you are wondering who the CA is and why you can trust them. This problem is address by public key infrastructure (PKI), a framework to manage certificates. [X.509 standard](#) defines the most commonly used format for public key certificates. The PKI model used by the internet contains multiple globally trusted third parties, and each party is a root CA. Branching out from each root CA is a tree of CAs, as in the below figure. The root CA issues certificates for its immediate children CAs. Any CA in the tree can issue certificates. Modern operating systems and web browsers are integrated with a collection of root certificates, which are trusted. These root certificates are used to validate the signature on the certificate issued by the root CA. With the signature verified, CA1, CA2, and CA3 can be trusted, so the signature on their issued certificates can also be verified. Eventually, CA11, which issues certificates to PayPal, can be trusted. If a certificate cannot be validated using PKI, your browser will present a security warning, indicating the risk of accessing that webpage. (Recall the warning we saw in our Lab 2.)



Test Yourself

Browse mobile apps on your smartphone. For those that require a login, see how many of them provide the option for delegated authentication. Who do they delegate the authentication to? After you log in using the delegated authentication, what else do the apps ask for? Are there any potential security problems?

Test Yourself

Check the certificate of a website you are browsing. Usually, on your web browser, there is small lock besides the URL like this: . Clicking the lock will allow you to review the site information, such as its certificate. Examine the certificate and identify the issuer, who this certificate is issued to, the validity of the certificate, and cryptographical information of the certificate.

■ Topic 2 Access Control

Introduction

Access control, which usually happens after authentication, is used to decide whether a **subject** can perform a requested **operation** on an **object**. Subjects are human users, often represented by surrogate programs running on behalf of the users. Objects are things on which an action can be performed, e.g., files, tables, programs, memory objects, hardware devices, strings, data fields, network connections, processors, etc. The operations performed on objects include, but are not limited to, read, write, modify, delete, execute, create, append, copy, etc. Decisions around allowing or denying the requested operations are based on access control policy.

Implementing Access Control

Access Control Matrix

One straightforward way to represent access control policy is to use an access control matrix, a table with each row representing a subject, each column representing an object, and each entry representing the access right for that subject on that object. We usually use “-” to indicate the subject does not have any access right on that object. For instance, in the example below, Process 3 can write File 1 and read File 4, but do nothing on other files. “Owner” means the process owns the file, i.e., is the creator of the file. Typically, the owner is authorized to change the rights of other subjects on the object. For instance, in the matrix below, Process 2 is the owner of File 3, so Process 2 can change/assign the rights of other processes on File 3, e.g., revoking read from Process 1 and assigning write to Process 3. Some systems also allow the owner to delegate the owner right to other subjects. For example, after Process 2 delegates the File 3 owner rights to Process 3, Process 3 will be able to change/assign the rights of other processes on File 3. Although the access control matrix is able to represent access control policy clearly, it suffers from the sparse matrix problem, i.e., most entries in the matrix are empty, especially for a complicated system with hundreds of processes and thousands of files.

Access Control Matrix					
	File 1	File 2	File 3	File 4	File 5
Process 1	read	read, write	read	read	-
Process 2	-	-	owner, read, write	-	read
Process 3	write	-	-	read	-

Access Control List and Capability List

The solution is to either use an **access control list (ACL)** or a **capability list**. The former can be thought of as separating the access control matrix based on columns and removing the empty entries. This results in a collection of lists, with each one representing all the subjects that have rights on an individual object. For instance, the first of the below figures shows the access control list for File 1. An ACL is associated with each object, listing the subjects and their allowed access types. In contrast, the capability list can be thought of as separating the access control matrix based on rows and removing the empty entries. This results in a collection of lists, with each one representing all the rights that an individual subject has on the objects. For instance, the second of the below figures shows the capability list for Process 1. A capability list is associated with each subject, listing the objects and the allowed access types on them. Both the ACL and capability list can address the sparse matrix problem well.

Access Control List for File 1	
	File 1
Process 1	read
Process 3	write

Capability List for Process 1				
	File 1	File 2	File 3	File 4
Process	read	read, write	read	read

If the access control policy can be modified by any user, e.g., the object owner changing the rights of other subjects on the object, we call it **discretionary access control (DAC)**. Unix/Linux mode bits and Windows file manager are examples of DAC. If the access control policy can only be modified by a trusted system or trusted system administrator, we call it **mandatory access control (MAC)**. Examples of MAC include the Bell-LaPadula model and Biba integrity model. We will discuss those examples later in this module.

Reference Monitor

Reference monitor is the classical approach for enforcing access control policy. It consists of three components: the reference monitor interface, authorization module, and policy store. The reference monitor interface defines where the authorization module should be invoked to make a specific authorization query to the policy store. The

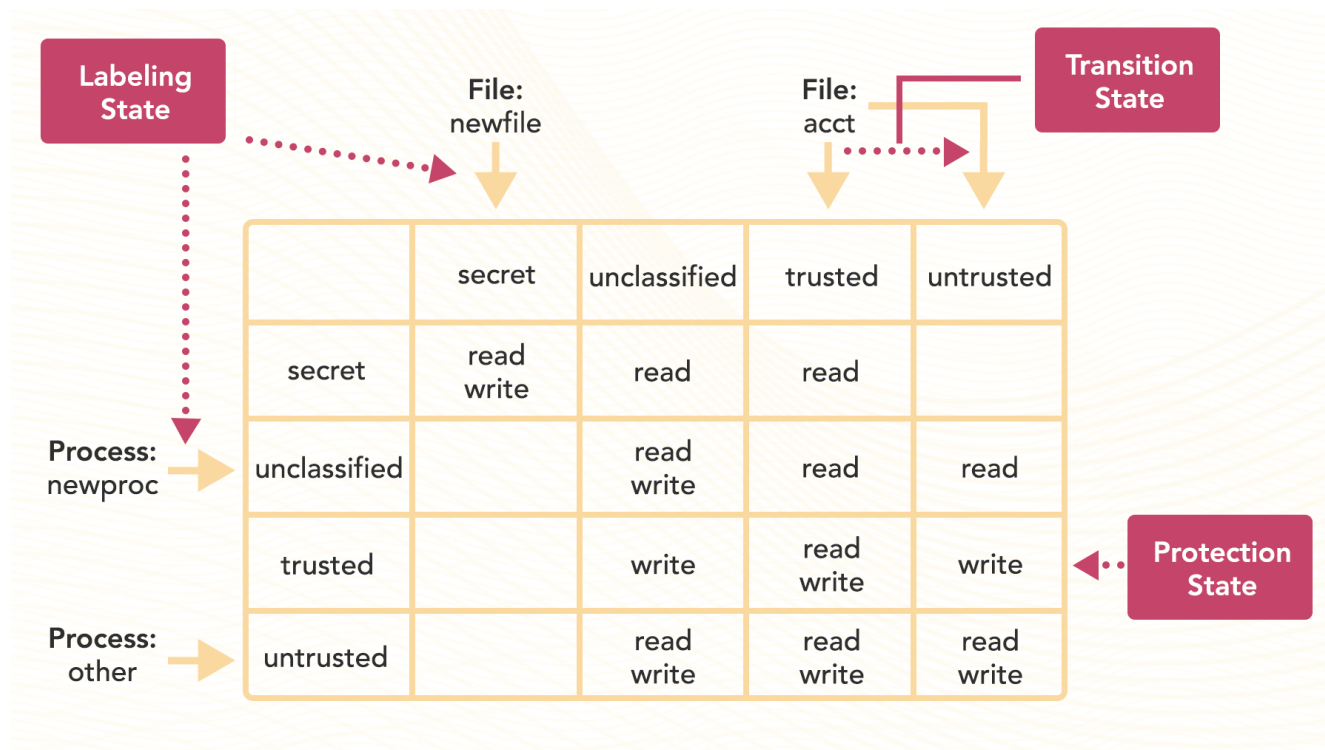
policy store contains the access control policy, and is thus able to respond to any authorization query. The reference monitor needs to provide three guarantees: that it provides complete mediation (it is always invoked on every access attempt), that it is tamperproof (immune from tampering), and that it is verifiable (assuredly correct).

Role-based Access Control (RBAC)

Role-based access control allows associate privileges/rights within groups, e.g., all employees of HR can access payroll information, all software test engineers can assess platform 1 and project 1, etc. Therefore, managing access control policy is easy, i.e., by changing the role for someone alongside his/her change of responsibility, rather than changing the policy setting for him/her separately. A classic example of RBAC is a database management system (DBMS). Refer to [Configuring Role-Based Access Control \(RBAC\)](#) for more details.

Protection System

The protection system manages the access control policy for the operating system, which includes the protection state, labelling state, and transition state. The labelling state specifies the policy of how to label newly created subjects and objects. With each subject and object labelled, the protection state is the policy that determines the allowed operations that subjects can perform on objects based on their label. Consider the matrix below. We have four labels: secret, unclassified, trusted, and untrusted. The row indicates the labels for subjects and the column indicates the labels for objects. The interpretation of the matrix is similar to the access control matrix we have seen above, with the difference being that the protection state is specified based on labels. For instance, based on the matrix, we know a subject labelled as secret can read and write an object labelled as secret. The transition state specifies the relabeling policy, that is, how to relabel subjects/objects given a specific operation just finishes. For instance, after a file with the trusted label is written by a process with the untrusted label, the file should be relabeled to untrusted.



From [os security, 1ed] (Jaeger, 2008)

A protection system is mandatory (a **mandatory protection system**) if and only if the protection state, labelling state, and transition state are all mandatory. If any of the three states is not mandatory, then the protection is discretionary (a **discretionary protection system**), not mandatory at all.

Problems of DAC

DAC cannot guarantee information secrecy or integrity, because it allows any untrusted subject to tamper with the access control policy. For instance, say our goal is to assure that Process 2 is not able to access any data in File 1 under any circumstance (e.g., File 1 contains highly confidential data, but Process 2 is suspicious). Suppose someone designed a set of policy rules and put it in the below access control matrix. The policy is DAC, i.e., file owners (even untrusted ones) can change the access rights of other processes on the file. Can such a policy achieve our desired assurance? At first glance, the answer seems to be yes. Based on the policy, Process 2 is not allowed to read File 1 directly. However, Process 2, the owner of File 3, can assign the write right of File 3 to Process 1, and simply wait for Process 1 to make mistakes. Allowed by the policy, Process 1 may read data from File 1 and incidentally write it to File 3. Then Process 2 can access such data, since it has the read right of File 3. Eventually, we can see the data in File 1 may first flow to Process 1, then File 3, and finally Process 2 indirectly. Such indirect data flow still compromises the assurance that, Process 2 is not able to access any data in File 1 under any circumstance. In summary, DAC cannot provide the assurance. Even if the policy is initially set up correctly, allowing untrusted subjects to modify the policy may compromise the assurance guarantee.

Access Control Matrix

	File 1	File 2	File 3	File 4	File 5
Process 1	read	read, write	read	read	-
Process 2	-	-	owner, read, write	-	read
Process 3	write	-	-	read	-

Unix Protection System

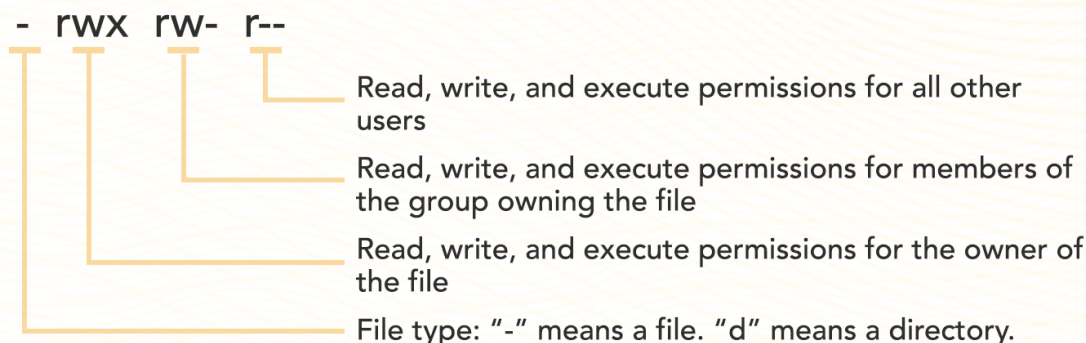
The traditional Unix protection system is straightforward. The subjects are processes (running programs), and objects are files. Each user, after logging in, will be assigned a user id (UID), a group id (GID), and a set of supplementary groups. The labelling state specifies (1) a process inherits UID and GID from its creator user; and (2) a file inherits UID and GID from its owner user. The possible operations on files include read, write, and execute (if the file is binary and can be run). Unix utilizes mode bits to implement the protection state. In the Unix/Linux terminal, typing “ls -l” will show the information of all files under the current directory, as shown on the left in the below figure. Refer to [Understanding ls command output](#) for an explanation of each field in the output.

In particular, we are interested in the first column of the output, which is the mode bits, as shown on the right in the below figure. The first field indicates this is a file (using “-”) or a directory (using “d”). Then, the next nine fields are information about the protection state. “r” indicates read permission, “w” indicates write permission, and “x” indicates execute permission. A “-” in a protection state indicates nothing, or no permission. The nine fields can be divided into 3 parts: owner, group, and all others. The first three fields are for the requester (the process that requests access to this file) whose UID matches the UID of this file. The middle three fields are for the requester (the process that requests access to this file) whose UID does not match the UID of this file, but whose GID matches the GID of this file. The last three fields are for the requester (the process that requests access to this file) whose UID or GID does not match this file, indicating all other processes. For instance, based on the information on the right in the below figure, a process with the same UID as the file can read, write, and execute the file; a process with different UID but the same GID as the file can read, and write the file; a process with different UID and different GID as the file can only read the file.

Unix Mode Bits

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

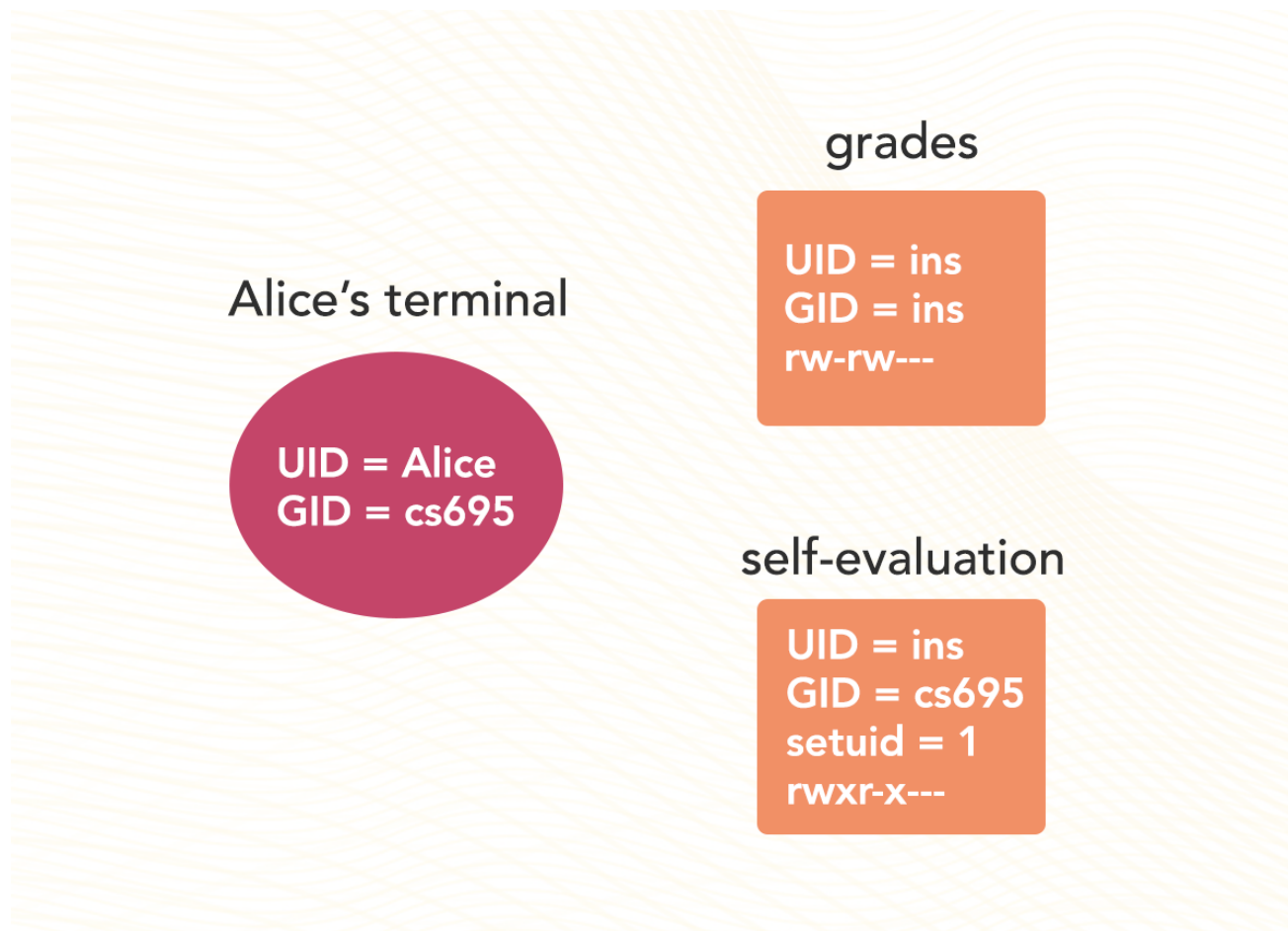
Interpretation of Mode Bits



Test Yourself

On a Linux computer (e.g., Kali Linux used in our lab), open a terminal and go to a directory with some files and subdirectories. Run the "ls -l" command to check the mode bits of all files and subdirectories. Explain the meaning of them.

The transition state is a little bit complicated. A classic example is the setuid bit, associated with an executable file. With this bit on (set by the owner of this executable file), the process that is executing this file will temporarily use the UID of the owner of this file, with its original UID disabled. The example illustrates the usage of the setuid bit. The instructor implements a self-evaluation program and allows each student to run it and evaluate their performance. The self-evaluation program, when running, needs to read a file called grades to access its data. Since the grades file contains confidential information for all students, only the instructor can read and write it (defined by its mode bits). Students, such as Alice, cannot read or write it. Based on the mode bits in the figure, the self-evaluation program allows Alice to run it in her terminal (GID matches), but the program cannot run correctly since Alice's terminal does not have the right to read data from the grades file, which is required by the self-evaluation program. If we simply change the GID of the grades file to be "cs695" (which matches Alice's), it will violate the confidentiality requirement, since the grades file contains confidential information for all students and we do not want individual students to be able to read it. The solution in this scenario to still allow Alice to run



the self-evaluation program is to use the setuid bit. The owner of the self-evaluation program can set its setuid bit on (assign it to be 1). This means any process, e.g., Alice's terminal, when executing the self-evaluation program, will temporarily use the UID of the self-evaluation program, i.e., "ins." With the temporary UID of "ins," Alice's terminal can read data from the grades file, thus allowing the self-evaluation program to run correctly. After running the self-evaluation program, Alice's terminal will resume its own UID, i.e., "Alice." From this example, we can see the setuid bit allows a process to temporarily change its label (UID) to that of the program file it is executing.

Stop and Research

The setuid bit is widely used in Unix/Linux systems to allow correct execution of programs. However, such **privilege escalation** (allowing the caller process to use the privileges of the program file owner) causes lots of security issues. Research these on your own and learn about setuid bit vulnerability.

Is the Unix protection system mandatory or discretionary? We must examine each of its states to see if they are mandatory. First is the protection state, i.e., the mode bits, can be modified by the file owner. On the Unix/Linux system, you can use the "chmod" command to change the mode bits of the file you created. Since file owners can just be any regular user, rather than system administrators, the protection state is not mandatory. Second, the labels of processes and files are based on their creators, so the labelling state satisfies the mandatory requirement. Finally, any program file owner, by setting the setuid bit on, can change the UID for the caller process to its own. Such a transition state is clearly not mandatory. Recall that a protection system is mandatory

if and only if all its states are mandatory. Since its protection state and transition state are not mandatory, the Unix protection system is not mandatory overall.

MAC Models

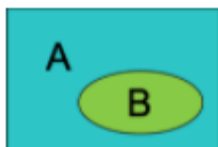
Bell-LaPadula Model

The Bell-LaPadula (BLP) model was developed by David Elliott Bell and Leonard J. LaPadula to enforce access control in government and military applications. It focuses on data confidentiality or secrecy. Each subject and object are assigned a security class, which represents two dimensions: sensitivity level and need-to-know. The sensitivity levels consist of the four governmental security classes: top-secret, secret, confidential, and unclassified, from the highest level to the lowest. However, a subject with a particular sensitivity level may not need to know all the information within that level, so BLP also defines a set of categories that describe the topic areas indicating need-to-know access. The BLP model assigns a sensitivity level that the subject is authorized for, and also a set of categories, called a compartment, to each subject and object. Below, we use s to indicate subject, o to indicate object, and $SC()$ to indicate the security class of a subject or an object.

1. The simple-security property (**read access**): s can read o only if $SC(s) \geq SC(o)$, i.e., the subject sensitivity level is greater than or equal to the object sensitivity level, and the subject compartment is a superset of object compartment.
2. The *-security property (**write access**): s can write o only if $SC(o) \geq SC(s)$, i.e., the object sensitivity level is greater than or equal to subject sensitivity level, and the object compartment is a superset of subject compartment

Implicitly, s can read and write o only if $SC(o) = SC(s)$, i.e., the subject sensitivity level is equal to the object sensitivity level, and the subject compartment is equal to the object compartment.

Superset



In mathematics, a set A is a superset of another set, B , if all elements of B are also elements of A . B is also called a subset of A . For example, if A is the set $\{\diamond, \heartsuit, \clubsuit, \spadesuit\}$ and B is the set $\{\diamond, \clubsuit, \spadesuit\}$, then A is a superset of B , since all elements of B are also in A .

Test Yourself

We use four sensitivity levels, top secret (TS), secret (SEC), confidential (CONF), and unclassified, to label subjects and objects. They are ordered from highest to lowest, i.e., $TS > SEC > CONF >$

unclassified. We have 3 categories, INTEL, CRYPTO, and NUC, which can be used to assign compartments to subjects and objects. We have three subjects and three objects labelled as below:

Alice: SEC, {NUC}

Bob: CONF, {INTEL}

Charlie: TS, {CRYPTO, NUC, INTEL}

DocA: CONF, {INTEL}

DocB: SEC, {CRYPTO, INTEL}

DocC: UNCLASSIFIED, {NUC}

Who can read which document(s), and who can write which document(s)?

Read:

Write:

The BLP model implements a mandatory protection system. First, its protection state is mandatory. The security classes including sensitivity levels and categories are defined by trusted administrators. The protection state depends on the dominance relationship among security classes, which is fixed based on the model. Second, its labelling state is mandatory. Trust administrators assign security classes to subjects and objects when they are created. Once the subject or object is created and labeled, its label is static. Third, the BLP model defines a null transition state, which means relabeling is not allowed. So, no one can change this null transition state. Overall, the BLP model satisfies the requirements for a mandatory protection system.

BLP can ensure data secrecy. Since the compartment determines need-to-know only, we only consider the sensitivity levels. Based on the simple-security property, a subject can read an object if its sensitivity level is greater or equal to that of the object. For the read operation, the data should flow from the object to the subject. If the subject and the object have the same sensitivity level, data just flows within that specific level, not outside. Otherwise, the data flows from the lower sensitivity level (the object) to the higher sensitivity level (the subject). Therefore, for the read operation, data is only allowed to flow from lower levels to higher levels based on the simple-security property. Based on the *-security property, a subject can write an object if its sensitivity level is less than or equal to that of the object. For the write operation, the data should flow from the subject to the object. If the subject and the object have the same sensitivity level, data just flows within that specific level, not outside. Otherwise, the data flows from the lower sensitivity level (the subject) to the higher sensitivity level (the object). Therefore, for the write operation, data is only allowed to flow from lower levels to higher levels based on the simple-security property as well. Overall, simple-security and *-security only allow data to flow from lower levels to higher levels, and there is no way for data to flow in the opposite direction. This implies that if we have confidential data, initially labelled as TS, by enforcing the BLP model, such confidential data will never be leaked to lower levels.



As you may have noticed, the BLP model does allow a lower-level subject to write/modify higher level objects, which may compromise the integrity of the higher-level object. As mentioned earlier, the BLP model only protects data confidentiality, and data integrity is out of its scope.

Biba Integrity Model

The BLP model only protects data secrecy, but the Biba Integrity model is designed to protect data integrity.

Each subject and object are assigned an integrity level. (There is no compartment unlike the BLP model.) There are no commonly agreed upon integrity levels, which means one can define the levels based on his/her own need. Below, we use *s* to indicate subject, *o* to indicate object, and *SC()* to indicate the security class of a subject or an object.

1. The simple-integrity property (**read access**): *s* can read *o* only if $SC(o) \geq SC(s)$, i.e., the object integrity level is greater than or equal to the subject integrity level
2. The *- integrity property (write access): *s* can write *o* only if $SC(s) \geq SC(o)$, i.e., the subject integrity level is greater than or equal to the object integrity level

Implicitly, *s* can read and write *o* only if $SC(o) = SC(s)$, i.e., the subject integrity level is equal to the object integrity level.

Like the BLP model, the Biba Integrity model is also a mandatory protection system. In contrast to the BLP model, data can only flow from higher integrity levels to lower integrity levels based on the Biba integrity model, thus protecting the integrity of data labelled with a higher integrity level.

Test Yourself

We use four sensitivity levels, top secret (TS), secret (SEC), confidential (CONF), and unclassified, to label subjects and objects. They are ordered based on their dominance, i.e., $TS > SEC > CONF > \text{unclassified}$. We have four integrity levels: trusted, application, user, untrusted. They are ordered based on their dominance, i.e., $\text{trusted} > \text{application} > \text{user} > \text{untrusted}$. We have one object and two subjects labelled as below.

Object: TS, user

Subject 1: TS, untrusted

Subject 2: CON, trusted

Consider a system that uses both BLP (we just ignore the compartment for simplicity) and Biba models to enforce access control, i.e., any access must be authorized by both BLP and Biba policies. Which subject can read and which subject can write the object?

Subject 1 can read the object

Subject 2 can write the object

References

- Pfleeger, C. P., Pfleeger, S. L., & Marguiles, J. (2015). *Security in computing* (5th ed.). Prentice Hall.
- Jaeger, T. (2008). *Operating system security* (1st ed.). Morgan and Claypool Publishers.

Boston University Metropolitan College